FH Salzburg
MultiMediaTechnology

Basics Game Development

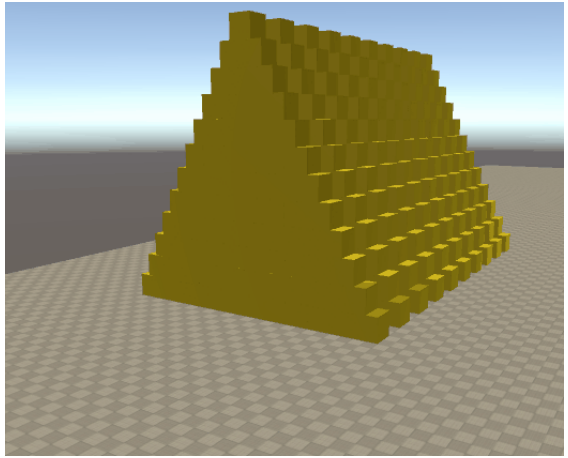
Collisions, Physics

Based on GD1 Slides by M. Tatzgern

Technology
Health
Media

Why Game Physics?

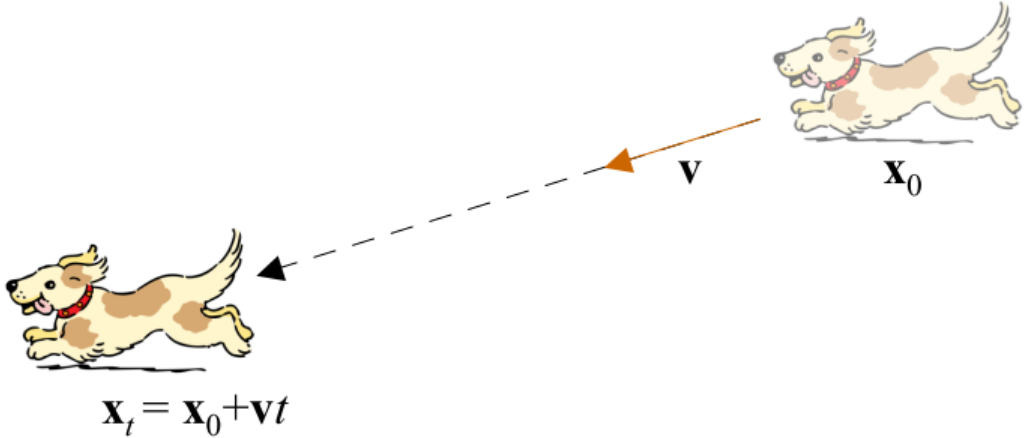
- Collision Detection (Trigger / Collider)
- Collision solving
- More realistic Movement and Animation



Physics-based motion

Constant Velocity (not physical)

Position at point in time known




```
Position += new Vector2f(0, 1) * moveSpeed * deltaTime;
```

Behaviour of a Physical System: Equations of Motion

Based on Netwon's Laws of motions

I) An object either remains at rest or continues to move at a constant velocity, unless acted upon by an external force (**inertia law, Trägheitsgesetz**).

II) The vector sum of the forces F on an object is equal to the mass m of that object multiplied by the acceleration vector a of the object: F = ma => a = F / m (**action law, Aktionsprinzip**)

III) When one body exerts a force on a second body, the second body simultaneously exerts a force equal in magnitude and opposite in direction on the first body (**reaction law, Reaktionsprinzip**).

Simplified for Physics in games

Rigid Body Definition

Position

current position

(Linear) Velocity

change of position over time

(Linear) Acceleration

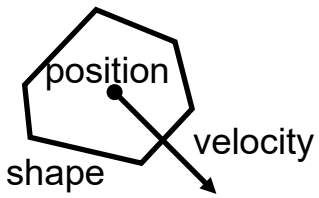
change of velocity over time

Shape

Defines the shape of the body

Mass

Object's resistance to acceleration
(Newton’s Second Law of Motion)



Equation of Motion in Games

Warning: Physics-based movement is not suitable for all game mechanics. Choose whisely!

```
class RigidBodyComponent
{
    Vector2f m_velocity;
    Vector2f m_position;
    void Update(float delta)
    {
      m_position += m_velocity * delta;
    }
}
```

Simplified version without acceleration

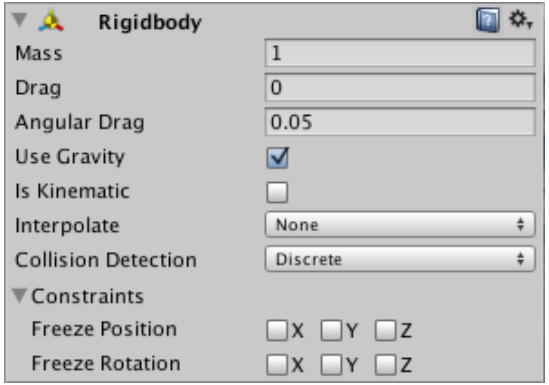
```
class RigidBodyComponent
{
    Vector2f m_acceleration;
    Vector2f m_velocity;
    Vector2f m_position;
    float m_mass;

    void Update(float delta)
    {
      Vec2f F = CalculateForces();

      // Symplectic Euler
      m_acceleration = F / m_mass;
      m_velocity += m_acceleration * delta;
      m_position += m_velocity * delta;
    }
}
```

Standard version common in games

Force Modes in Unity

Force	Add a continuous force to the rigidbody, using its mass.
Acceleration	Add a continuous acceleration to the rigidbody, ignoring its mass.
Impulse	Add an instant force impulse to the rigidbody, using its mass.
VelocityChange	Add an instant velocity change to the rigidbody, ignoring its mass.

Physics Component in Unity



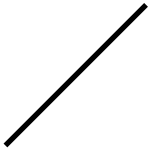
Collision Detection

Pixel Based Collision Detection

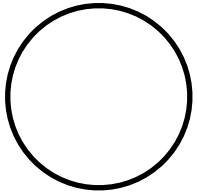
- Test every pixel for intersection
  → Perfect collision detection.
- Very time consuming!
  → Used only for mouse hit test in 2d games.
- Use approximation instead:
  → AABB, circle or convex polygon
  → Compound multiple shapes if necessary

Collision Shapes 2D (Collider)

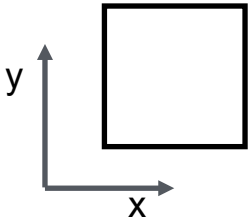
Point



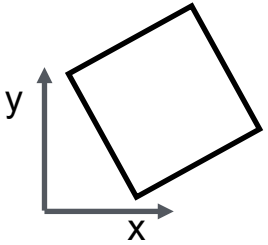
Line



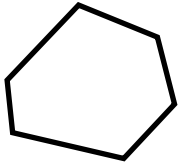
Circle



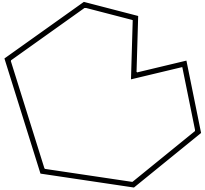
Axis-Aligned Bounding Box (AABB)



Oriented Bounding Box (OBB)

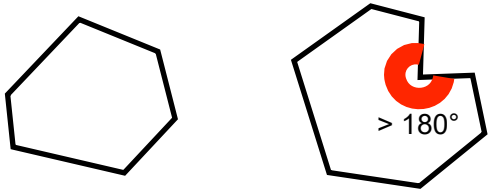


Convex Polygon



Concave Polygon

Convex vs Concave



Convex Polygon Concave Polygon

Convex Polygon

Every internal angle is strictly less than 180 degree

Concave

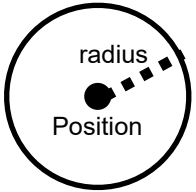
At least one internal angle is >= 180 degree

Problematic for collision detection.

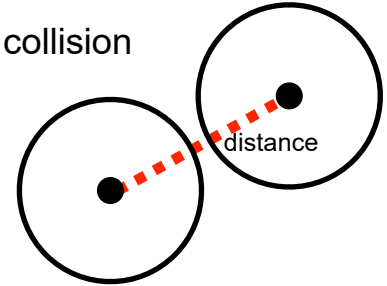
Most engines don’t support concave polygons.

Concave polygon can be partitioned into a set of convex polygons.

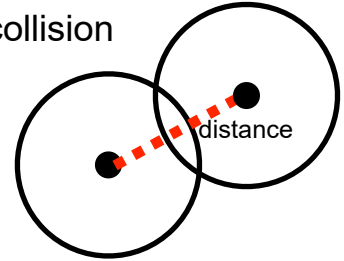
CIRCLE



no collision



has collision



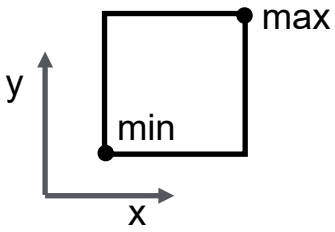

```
struct Circle
{
    public Vector2f Position {get; set;}
    public float Radius {get; set;}
};
```

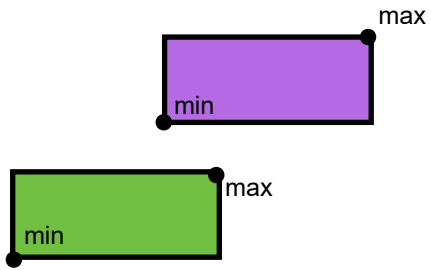
Circle definition

```
float Distance(Vector2f a, Vector2f b)
{
    return sqrt((a.x - b.x) ^ 2 + (a.y - b.y) ^ 2)
}

bool CircleVsCircle(Circle a, Circle b)
{
    float r = a.radius + b.radius;
    return r < distance(a.position, b.position);
}

// Optimized version without square root
bool CircleVsCircleOptimized(Circle a, Circle b)
{
    float r = a.radius + b.radius;
    return r * r < (a.x - b.x) ^ 2 + (a.y - b.y) ^ 2;
}
```

Shortcut for illustration purposes

AXIS ALIGNED BOUNDING BOX



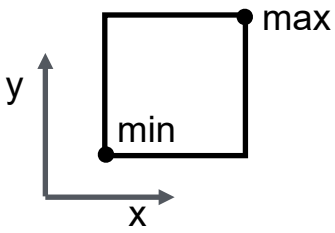


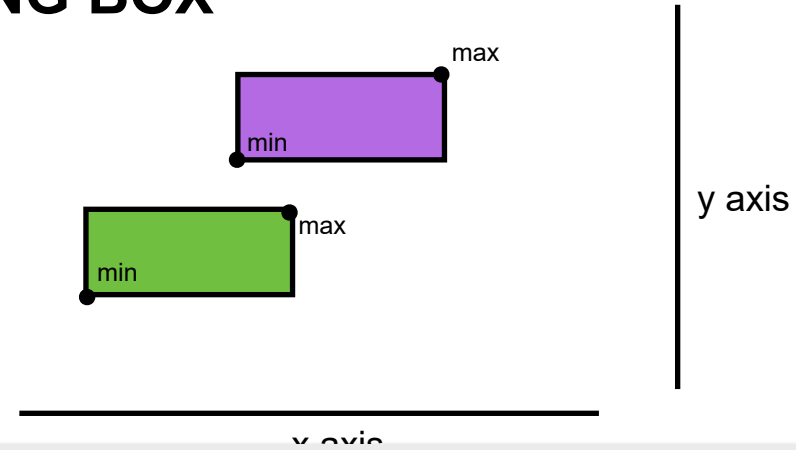

```
struct AABB
{
    Vector2f min;
    Vector2f max;
}
```

AABB definition

```
bool AABBvsAABB(AABB a, AABB b)
{
    /??
}
```

AXIS ALIGNED BOUNDING BOX



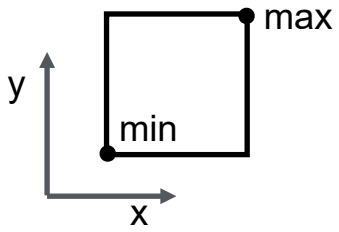


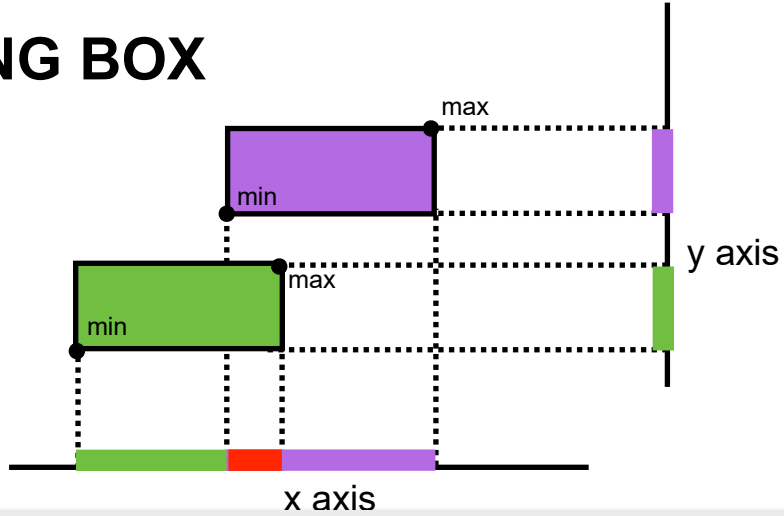

```
struct AABB
{
    Vector2f min;
    Vector2f max;
}
```

AABB definition

```
bool AABBvsAABB(AABB a, AABB b)
{

    Separating Axis Theorem:
    Check if there is at least one separating axis

}
```

AXIS ALIGNED BOUNDING BOX



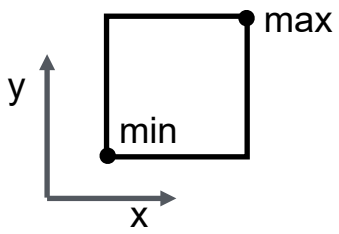


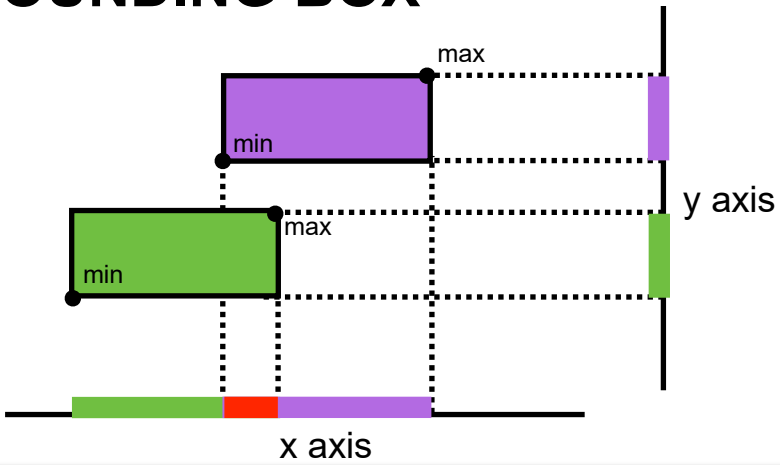

```
struct AABB
{
    Vector2f min;
    Vector2f max;
}
```

AABB definition

```
bool AABBvsAABB(AABB a, AABB b)
{

    no collision if there is no overlap on at least one axis

}
```

AXIS ALIGNED BOUNDING BOX



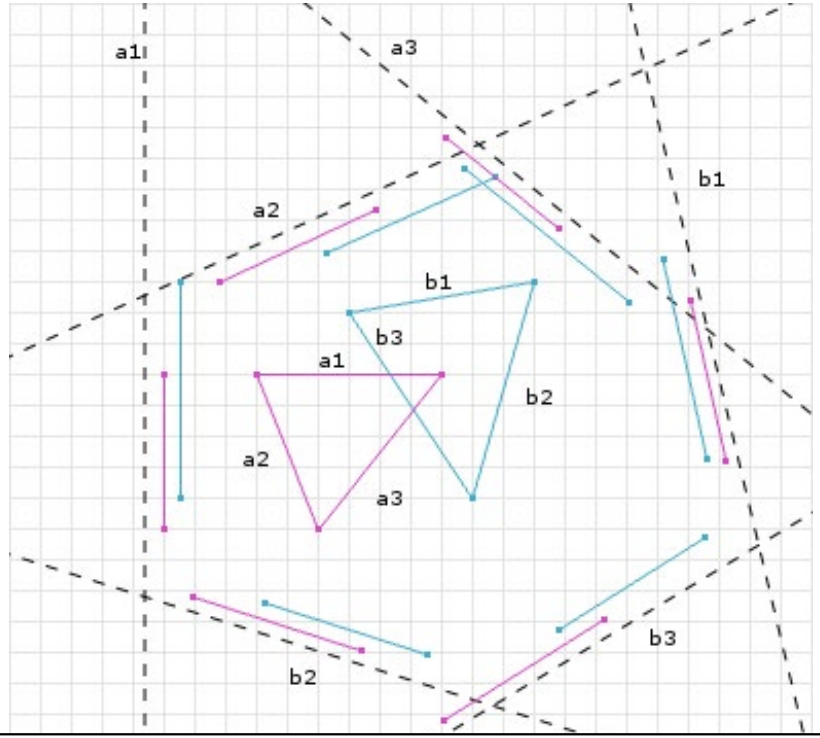



```
struct AABB
{
    Vector2f min;
    Vector2f max;
}
```

AABB definition

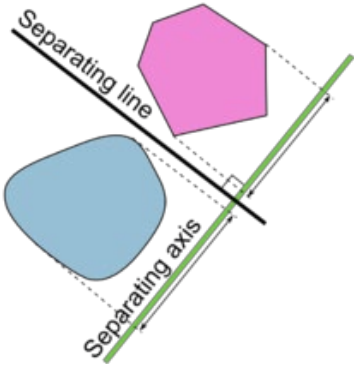
```
bool AABBvsAABB(AABB a, AABB b)
{
    // Exit with no intersection if found separated along an axis
    if (a.max.x < b.min.x || a.min.x > b.max.x) {
        return false;
    }
    if (a.max.y < b.min.y || a.min.y > b.max.y) {
        return false;
    }

    // No separating axis found, -> we have a collision
    return true;
}
```

Separating Axis Theorem

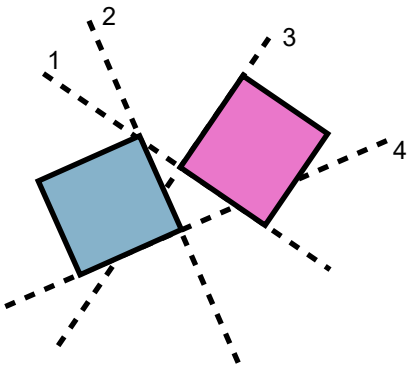


General SAT explained: http://www.dyn4j.org/2010/01/sat/

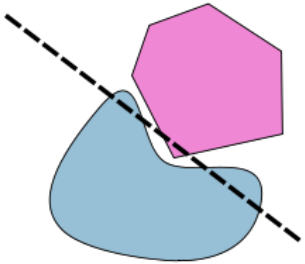
Separating Axis Theorem



Convex objects



OBB in 2D need
maximum 4 axis checked



SAT doesn't work for
Non-convex objects

Two convex objects do NOT collide (are separated) if there exists one (separating) axis on which projections of objects are separated

That means: Run through checks until you find 1 axis without overlap.

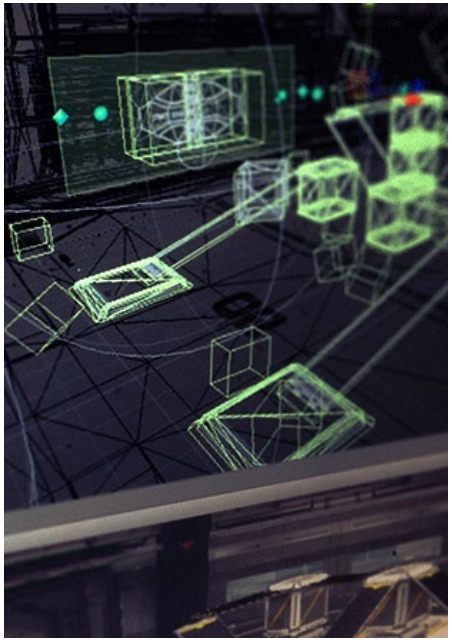
Circle vs AABB Collision

```
bool Circle_AABB_Collision(CircleShape circle, RectangleShape rect)
{
    var getCenter = (Shape shape)=>{return new Vector2f(){
        X= shape.GetGlobalBounds().Left + shape.GetGlobalBounds().Width / 2f,
        Y= shape.GetGlobalBounds().Top + shape.GetGlobalBounds().Height / 2f};};

    Vector2f circleDistance = new();
    var centerCircle = getCenter(circle);
    var centerRect = getCenter(rect);

    circleDistance.X = MathF.Abs(centerCircle.X - centerRect.X);
    circleDistance.Y = MathF.Abs(centerCircle.Y - centerRect.Y);

    if (circleDistance.X> (rect.Size.X / 2 + circle.Radius)) { return false; }
    if (circleDistance.X > (rect.Size.Y / 2 + circle.Radius)) { return false; }

    if (circleDistance.X <= (rect.Size.X / 2)) { return true; }
    if (circleDistance.Y <= (rect.Size.Y / 2)) { return true; }

    var cornerDistance_sq = (circleDistance.X - rect.Size.X / 2) * (circleDistance.X - rect.Size.X / 2) +
                            (circleDistance.Y - rect.Size.Y / 2) * (circleDistance.Y - rect.Size.Y / 2);

    return (cornerDistance_sq <= (circle.Radius *circle.Radius));
}
```

Collision Resolving

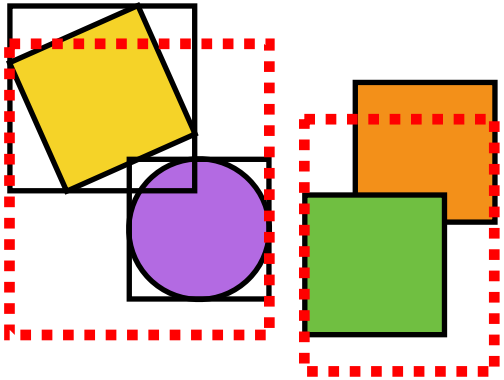
Collision Handling in Physics Engines

1. **Collision Detection**
   1. **Broadphase**
      → Find potential collisions
   2. **Narrow Phase**
      → Exact collision test
      → Calculate Manifold
2. **Collision Handling**
      → Resolve Collision
      → Positional Correction
3. **Integrate equations of motion**
      →Update the position of objects based on the velocity.



Broad Phase

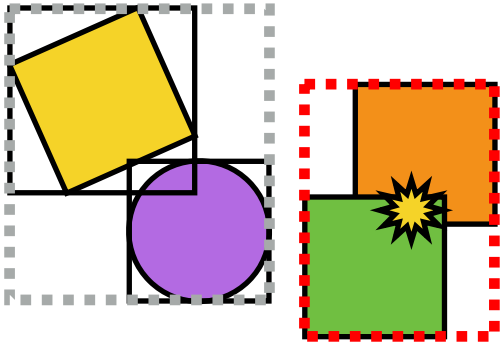
- Find potential collisions pairs.
- Not accurate but fast.
  → Use approximated AABB or Circle for each object.
- Can be optimized by grouping objects using:
  → Binary Space Partition Tree
  → Quad-Tree (2D)
  → Oct-Tree (3D)
  → Hierarchical Volumes
  → Sweep and Prune
  → Spatial Hashing



finds potential collisions using approximated AABBs

Narrow Phase

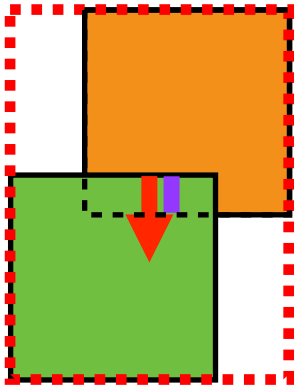
- Does an exact collision detection for every pair from broad phase.
  - Polygon based.
  - Pixel based.
- Creates a Manifold for every correct collision.
- Manifold is later used by the collision handling.



actual existing collisions are determined during the narrow phase

Manifold

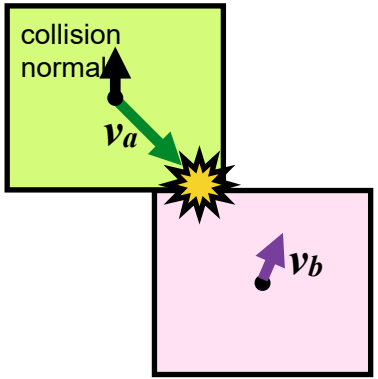
- Stores information about the collision needed for the collision handling
- Collision Normal
- Penetration Depth
    - Level of overlap between both objects
    - Amount we have to push the objects along the collision normal to resolve collision
- Colliding objects
- Contact Points (if needed)



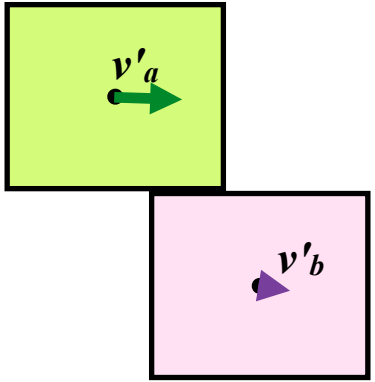
Resolving Collisions

Calculate one time impulse so that both objects are heading away in the next frame based on:

1. Collision Normal
2. Velocity of both objects



Velocity in current frame

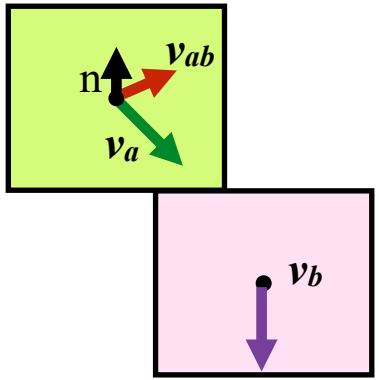


Velocity in next frame

Relative Velocity

Relative Velocity v_{ab} is the velocity of object A relative to object B

$$v_{ab} = v_a - v_b$$



Relative Velocity - Object Heading Away

Relative Velocity is the velocity of object A relative to object B

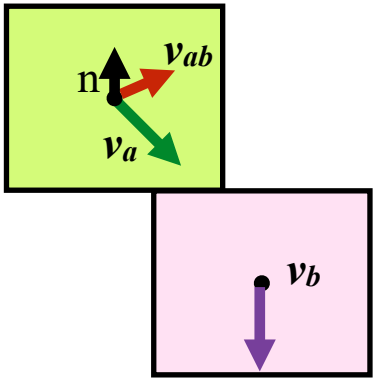
$$v_{ab} = v_a - v_b$$

If both relative velocity and collision normal point away from the other object, there is no collision.

→vector projection onto normal

$$v_{ab} \cdot n > 0$$

Ignore collision!



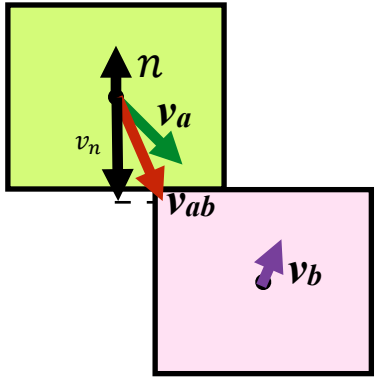
Objects heading away from each other

Relative Velocity Along Collision Normal

The portion of the relative velocity between two objects along the collision normal

→ Use this portion to calculate collision response:

$$v_n = (v_{ab} \cdot n)\, n$$

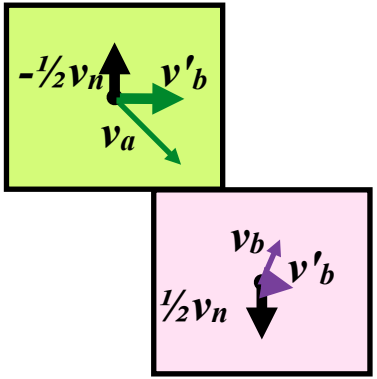


Simple Impulse Response – Relative Velocity

With the v_n we can push our objects apart.

Subtract half of v_n from v_a

Add half of v_n to v_b



v_a and v_b in current frame

v'_a and v'_b in next frame

$$v_n = (v_{ab} \cdot n)\, n$$

$$v'_a = v_a - \frac{1}{2} v_n$$

$$v'_b = v_b + \frac{1}{2} v_n$$

Resolve Collisions in Code (Version 1 C++)

```
void Physics::resolveCollision(Shape* a, Shape* b, ci::Vec2f normal)
{
    // Calculate relative velocity
    ci::Vec2f rv = b->getVelocity() - a->getVelocity();

    // Calculate relative velocity in terms of the normal direction
    float velAlongNormal = rv.dot(normal);

    // Do not resolve if velocities are separating
    if (velAlongNormal < 0)
        return;

    // Apply impulse
    ci::Vec2f impulse = velAlongNormal * normal;

    a->setVelocity(a->getVelocity() + 0.5f * impulse);
    b->setVelocity(b->getVelocity() - 0.5f * impulse);
}
```

The Road to J (with mass)

Restitution formula

$$(v'_a - v'_b) \cdot n = -\varepsilon\,(v_a - v_b) \cdot n$$

Newton's second law

$$v'_a = v_a + j\frac{1}{m_a}n$$

$$v'_b = v_b - j\frac{1}{m_b}n$$

Intermediate step

$$(v_a + \frac{j}{m_a}n - v_b + \frac{j}{m_b}n) \cdot n = -\varepsilon\,(v_a - v_b) \cdot n$$

Now we can calculate j

$$j = \frac{-(1+\varepsilon)v_{ab} \cdot n}{\left(\frac{1}{m_a} + \frac{1}{m_b}\right)}$$

Final Resolve Collision Version in Code (C#)

```
void ResolveCollision(IRigidBody a, IRigidBody b, Vector2f normal)
{
    if (a.InverseMass == 0 && b.InverseMass == 0)
    {
        // If both bodies don’t have a mass it’s pointless to resolve the collision
        return;
    }

    // Calculate relative velocity
    Vector2f rv = a.Velocity - b.Velocity;

    // Calculate relative velocity in terms of the normal direction
    float velAlongNormal = rv.X * normal.X + rv.Y * normal.Y;


    // Do not resolve if velocities are separating
    if (velAlongNormal > 0)
        return;

    // For now we use a hard coded restitution.
    // If two objects have different restitutions, use the smallest one of the two (e = min( e1, e2))
    float e = 0.1f;

    // Calculate impulse scalar
    float j = -(1 + e) * velAlongNormal;
    j /= a.InverseMass + b.InverseMass;

    // Apply impulse
    Vector2f impulse = j * normal;
    a.Velocity = a.Velocity + a.InverseMass * impulse;
    b.Velocity = b.Velocity - b.InverseMass * impulse;
}
```

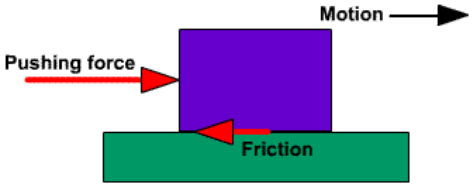
What else?

Simulation

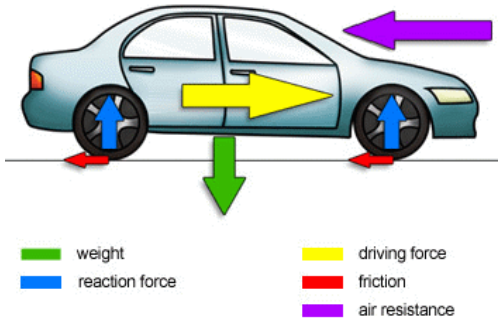
- Physics Engine needs a constant update rate.
- A variable update time is very bad for the physics calculations.
- Use accumulator to decouple physics update from the game

```
void Update(float delta)
{
    // define our physics update rate
    const float step = 1.0f / 100.0f;

    // m_accumulator is member of physics
    m_accumulator += delta;

    while (m_accumulator > step)
    {
        // update physics with a fixed step
        updateStep(step);
        m_accumulator -= step;
    }
}

void UpdateStep(float delta)
{
    // do the actual pyhsics update
}
```

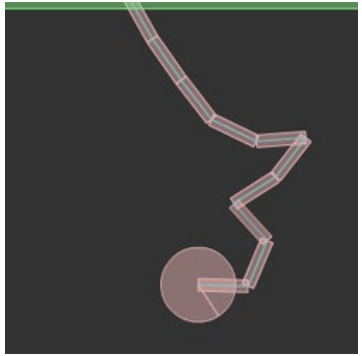
Friction, Forces and joints



Friction - Force when two objects slide upon each other



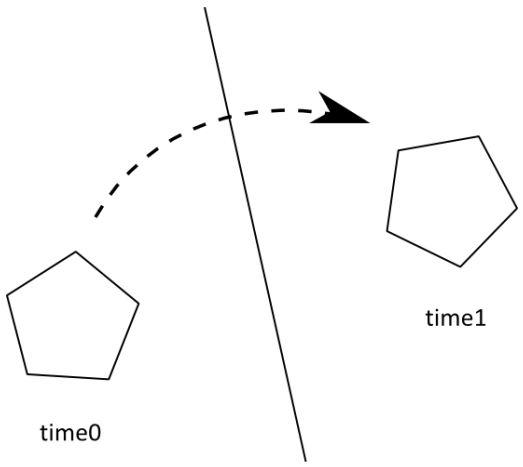
Different forces act on the vehicle



Bodies can be connected to each other using joints (or constraints)

Tunneling

- Physic engines are discrete systems
- Collision detection is only done at certain steps in time
- Fast objects can tunnel through narrow objects
- Solutions:
    - Decrease time step
    - Use continuous collision detection (e.g., sweeping volume)



Tunneling Effect

No collision is detected at time0 and time1

Physics SDK

Due to mathematical complexity physics libraries are very recommended.

Commercial:

- Havok
- PhysX

Open Source:

- Bullet
- ODE (old)
- Box2d (2D)

FH Salzburg
MultiMediaTechnology

Basics Game Development

Collisions, Physics - End

Technology
Health
Media

© 2025 Fachhochschule Salzburg GmbH
www.fh-salzburg.ac.at